TP 2 : le lakehouse ShopFlow sur Databricks

Travail en binôme. Plateforme : **Databricks Free Edition**.
Prérequis : TP 1 terminé côté Databricks (données SF1 dans le volume `workspace.shopflow.raw` et tables `orders`, `customers` dans `workspace.shopflow`).

> Contexte : jusqu'ici, ShopFlow recevait ses données sous forme de tables propres. En réalité, les paiements arrivent en fichiers CSV chaque mois, la navigation en JSON, et la banque envoie des fichiers de **rétrofacturations** (*chargebacks*) qui révèlent, des semaines plus tard, quels paiements étaient frauduleux. Vous allez construire un lakehouse en trois couches qui absorbe ces fichiers, en garantit la qualité et produit la table de variables (*features*) du futur modèle de fraude.

Conventions : tout est dans le catalogue `workspace`, schéma `shopflow`. Créez un dossier `tp2` dans votre *Workspace* avec un notebook par partie.

Partie 0 : simuler l'arrivée des fichiers

Les fichiers sources n'existent pas encore : un notebook les fabrique à partir des données du TP 1, **sans les colonnes que l'entreprise ne connaît pas au moment du paiement** (`is_fraud`, `fraud_scenario`, `chargeback_ts`), et avec quelques défauts volontaires (doublons, montants négatifs, identifiants manquants, lignes JSON corrompues).

1. Créez les volumes :

```
CREATE VOLUME IF NOT EXISTS workspace.shopflow.landing;
CREATE VOLUME IF NOT EXISTS workspace.shopflow.checkpoints;
```

2. Créez le notebook Python `00_landing` et copiez-y ce code (fourni, ne pas modifier) :

Partie 0 : simuler l'arrivée des fichiers (suite)

```python
import glob, os
import numpy as np
import pandas as pd

RAW = "/Volumes/workspace/shopflow/raw/sf1"
LAND = "/Volumes/workspace/shopflow/landing"
for d in ["payments", "chargebacks", "clickstream"]:
    os.makedirs(f"{LAND}/{d}", exist_ok=True)

HIDDEN = ["is_fraud", "fraud_scenario", "chargeback_ts"]

def land_payments(year: int) -> None:
    """Un fichier CSV par mois, avec doublons et lignes invalides."""
    for f in sorted(glob.glob(f"{RAW}/payments/part-{year}-*.parquet")):
        month = f.split("part-")[1][:7]
        df = pd.read_parquet(f).drop(columns=HIDDEN)
        dup = df.sample(frac=0.005, random_state=1)                    # renvois en double
        neg = df.sample(frac=0.001, random_state=2).assign(            # montants négatifs
            amount=lambda d: -d.amount, payment_id=lambda d: d.payment_id + 10_000_000)
        noid = df.sample(frac=0.0005, random_state=3).assign(payment_id=pd.NA)  # identifiant manquant
        out = pd.concat([df, dup, neg, noid]).sample(frac=1, random_state=4)
        out.to_csv(f"{LAND}/payments/payments_{month}.csv", index=False)
```

Partie 0 : simuler l'arrivée des fichiers (suite)

```
        print("écrit", month, len(out))

def land_chargebacks(start: str, end: str, name: str) -> None:
    """Rétrofacturations reçues entre start (inclus) et end (exclu)."""
    cols = ["payment_id", "chargeback_ts"]
    cb = pd.concat(pd.read_parquet(f, columns=cols) for f in glob.glob(f"{RAW}/payments/*.parquet"))
    cb = cb.dropna(subset=["chargeback_ts"])
    lo, hi = pd.Timestamp(start, tz="UTC"), pd.Timestamp(end, tz="UTC")
    cb = cb[(cb.chargeback_ts >= lo) & (cb.chargeback_ts < hi)].assign(reason="fraud")
    cb.to_csv(f"{LAND}/chargebacks/chargebacks_{name}.csv", index=False)
    print("écrit", name, len(cb))

def land_clicks(month: str) -> None:
    """Un mois de navigation en JSON lines, avec 100 lignes corrompues."""
    ev = pd.read_parquet(f"{RAW}/clickstream/part-{month}.parquet")
    path = f"{LAND}/clickstream/clicks_{month}.json"
    # Un volume n'accepte pas l'ajout en fin de fichier (mode "a") :
    # on prépare tout le texte en mémoire, puis on l'écrit en une seule fois.
    texte = ev.to_json(orient="records", lines=True, date_format="iso").rstrip("\n") + "\n"
    texte += '{"event_id": "abc", "event_ts": "pas-une-date", "event_type": \n' * 100
    with open(path, "w") as fh:
        fh.write(texte)
```

Partie 0 : simuler l'arrivée des fichiers (suite)

```
    print("écrit", month, len(ev))

# Premier lot : l'année 2024
land_payments(2024)
land_chargebacks("2024-01-01", "2025-01-01", "2024")
land_clicks("2024-12")
```

3. Vérifiez dans *Catalog > workspace > shopflow > landing* : 12 fichiers de paiements, 1 de rétrofacturations, 1 de navigation.

❓ Q0. Pourquoi a-t-on retiré `is_fraud` des fichiers de paiement ? Que représente `chargeback_ts` dans la vraie vie ?

Partie 1 : bronze, ingestion incrémentale

La couche bronze conserve les données **telles que reçues**, toutes les colonnes en texte, plus deux colonnes techniques : l'heure d'ingestion et le fichier d'origine.

1. Notebook `01_bronze`. Copiez cette fonction (fournie) :

Partie 1 : bronze, ingestion incrémentale (suite)

```python
from pyspark.sql import functions as F

LAND = "/Volumes/workspace/shopflow/landing"
CHK = "/Volumes/workspace/shopflow/checkpoints"

def ingest(source: str, fmt: str) -> None:
    """Ingère les NOUVEAUX fichiers de landing/<source> dans bronze_<source>, puis s'arrête."""
    stream = (spark.readStream.format("cloudFiles")
              .option("cloudFiles.format", fmt)
              .option("cloudFiles.schemaLocation", f"{CHK}/{source}/schema")
              .option("header", "true")
              .load(f"{LAND}/{source}")
              .withColumn("_ingested_at", F.current_timestamp())
              .withColumn("_source_file", F.col("_metadata.file_path")))
    query = (stream.writeStream
             .option("checkpointLocation", f"{CHK}/{source}/checkpoint")
             .trigger(availableNow=True)          # seul mode accepté sur le calcul serverless
             .toTable(f"workspace.shopflow.bronze_{source}"))
    query.awaitTermination()
```

Partie 1 : bronze, ingestion incrémentale (suite)

2. Ingérez les trois sources : `ingest("payments", "csv")`, `ingest("chargebacks", "csv")`, `ingest("clickstream", "json")`.
3. Inspectez : `DESCRIBE TABLE workspace.shopflow.bronze_payments;` puis comptez les lignes par `_source_file`.
4. **Relancez** `ingest("payments", "csv")` et recomptez.

❓ Q1. Quel est le type des colonnes de `bronze_payments` ? Pourquoi est-ce un choix raisonnable pour une couche bronze ?

❓ Q2. Que s'est-il passé à la deuxième exécution ? Quel mécanisme l'explique ?

❓ Q3. Dans `bronze_clickstream`, que sont devenues les 100 lignes corrompues ? (Cherchez les lignes où `event_type` est nul et regardez la colonne `_rescued_data`.) Quelle option d'Auto Loader permettrait de les isoler dans des fichiers à part ?

Partie 2 : silver, qualité et `MERGE`

Notebook `02_silver` (SQL ou Python avec `spark.sql`). Commencez par `USE workspace.shopflow;`.

Partie 2 : silver, qualité et `MERGE` (suite)

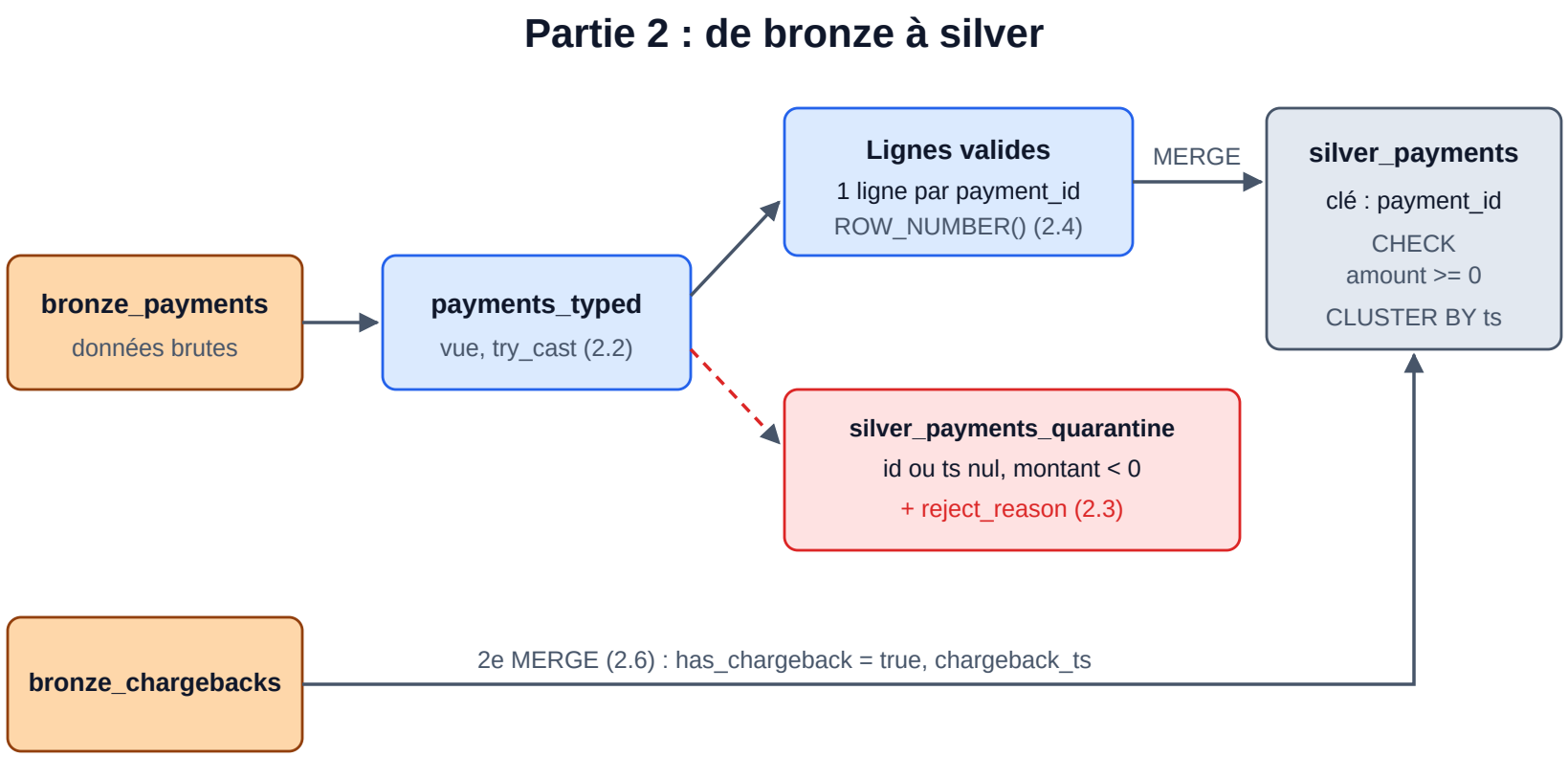


Schéma : les étapes 2.2 à 2.6 ; les numéros renvoient aux sous-parties ci-dessous.

2.1 Créer la table silver

```
CREATE TABLE IF NOT EXISTS silver_payments (
  payment_id BIGINT NOT NULL,
  order_id BIGINT,
  customer_id BIGINT,
  card_id BIGINT,
  payment_ts TIMESTAMP NOT NULL,
  amount DOUBLE,
  currency STRING,
  payment_method STRING,
  card_country STRING,
  ip STRING,
  status STRING,
  has_chargeback BOOLEAN,
  chargeback_ts TIMESTAMP
) CLUSTER BY (payment_ts);

ALTER TABLE silver_payments ADD CONSTRAINT amount_positive CHECK (amount >= 0);
```

2.2 Typer

Écrivez une vue temporaire `payments_typed` qui convertit chaque colonne de `bronze_payments` au bon type avec `try_cast` (qui renvoie `NULL` au lieu d'échouer), et conserve `_ingested_at`.

2.3 Mettre en quarantaine

Créez (ou complétez) `silver_payments_quarantine` avec les lignes de `payments_typed` dont `payment_id` ou `payment_ts` est nul, ou dont le montant est négatif, en ajoutant une colonne `reject_reason`.

2.4 Dédoublonner et fusionner

Les lignes valides peuvent contenir des doublons (même `payment_id`). Gardez une seule ligne par `payment_id` (indice : `ROW_NUMBER() OVER (PARTITION BY payment_id ORDER BY _ingested_at)`), puis insérez-les dans `silver_payments` avec un **`MERGE`** sur `payment_id`, en initialisant `has_chargeback` à `false`.

2.5 Tester

- Essayez d'insérer une ligne avec un montant négatif directement dans `silver_payments`. Que se passe-t-il ?
- Relancez **tout** le notebook. Le nombre de lignes de `silver_payments` change-t-il ?

Points de contrôle : `silver_payments` contient exactement le nombre de paiements de 2024 du TP 1 (comparez avec `SELECT COUNT(*) FROM payments WHERE year(payment_ts) = 2024`). La quarantaine contient environ 0,15 % des lignes reçues.

❓ Q4. Pourquoi un `MERGE` plutôt qu'un `INSERT` ? Utilisez le mot « idempotent ».
❓ Q5. Quel est l'intérêt de la quarantaine par rapport à une contrainte qui fait échouer l'écriture ?

2.6 Appliquer les rétrofacturations

Écrivez un deuxième `MERGE` qui, pour chaque ligne de `bronze_chargebacks`, passe `has_chargeback` à `true` et renseigne `chargeback_ts` dans `silver_payments`, seulement si ce n'est pas déjà fait.

Partie 3 : deuxième lot et *time travel*

1. Dans `00_landing`, ajoutez et exécutez :

```
land_payments(2025)
land_chargebacks("2025-01-01", "2026-04-01", "2025")
```

2. Relancez `01_bronze` (les trois ingestions) puis `02_silver` en entier.

3. Observez l'historique :

```
DESCRIBE HISTORY workspace.shopflow.silver_payments;
```

4. Trouvez la version de la table **juste avant** le `MERGE` des rétrofacturations de 2025, puis comparez :

Partie 3 : deuxième lot et *time travel* (suite)

```sql
SELECT COUNT_IF(has_chargeback) AS fraudes_connues
FROM workspace.shopflow.silver_payments VERSION AS OF <n>
WHERE payment_ts >= '2024-12-01' AND payment_ts < '2025-01-01';
-- même requête sur la version courante
```

❓ Q6. Pourquoi le nombre de fraudes connues pour décembre 2024 augmente-t-il entre les deux versions ?

❓ Q7. Un data scientist a entraîné un modèle le 1er janvier 2025. Comment retrouver exactement les données qu'il a vues ? Pourquoi est-ce important ?

Partie 4 : gold, la table de features

Le futur modèle (séance 5) doit décider, au moment d'un paiement, s'il est suspect, **en n'utilisant que le passé**. Construisez `gold_payment_features` : une ligne par paiement de `silver_payments`, avec les colonnes suivantes.

Partie 4 : gold, la table de features (suite)

Feature	Définition
`card_nb_tx_1d`, `card_nb_tx_7d`	Nombre de paiements de la même carte dans les 1 et 7 derniers jours (paiement courant inclus)
`card_avg_amount_7d`, `card_avg_amount_30d`	Montant moyen de la carte sur 7 et 30 jours
`amount_ratio_30d`	`amount / card_avg_amount_30d`
`ip_nb_tx_1h`	Nombre de paiements depuis la même IP dans la dernière heure
`ip_nb_cards_1h`	Nombre de cartes distinctes utilisées depuis la même IP dans la dernière heure
`account_age_days`	Âge du compte client au moment du paiement (`-1` pour un invité)
`ship_country_mismatch`	Pays de livraison différent du pays du client
`is_guest`	Paiement sans compte (`customer_id = 0`)
`label`	`has_chargeback` (le label observé)

Partie 4 : gold, la table de features (suite)

Rappel : la fenêtre glissante sur le temps. Pour chaque paiement, on regarde les paiements **de la même carte** dans les 7 jours **précédents**, et on les compte :

Partie 4 : gold, la table de features (suite)

```
COUNT(*) OVER (PARTITION BY card_id                          -- même carte
               ORDER BY unix_timestamp(payment_ts)           -- dans l'ordre du temps
               RANGE BETWEEN 604800 PRECEDING AND CURRENT ROW)   -- 7 jours avant, jusqu'au paiement courant
```

Ce sont les variables « comportement de dépense du client » du *Fraud Detection Handbook*.

Partie 4 : gold, la table de features (suite)

Fenêtre glissante de 7 jours par carte

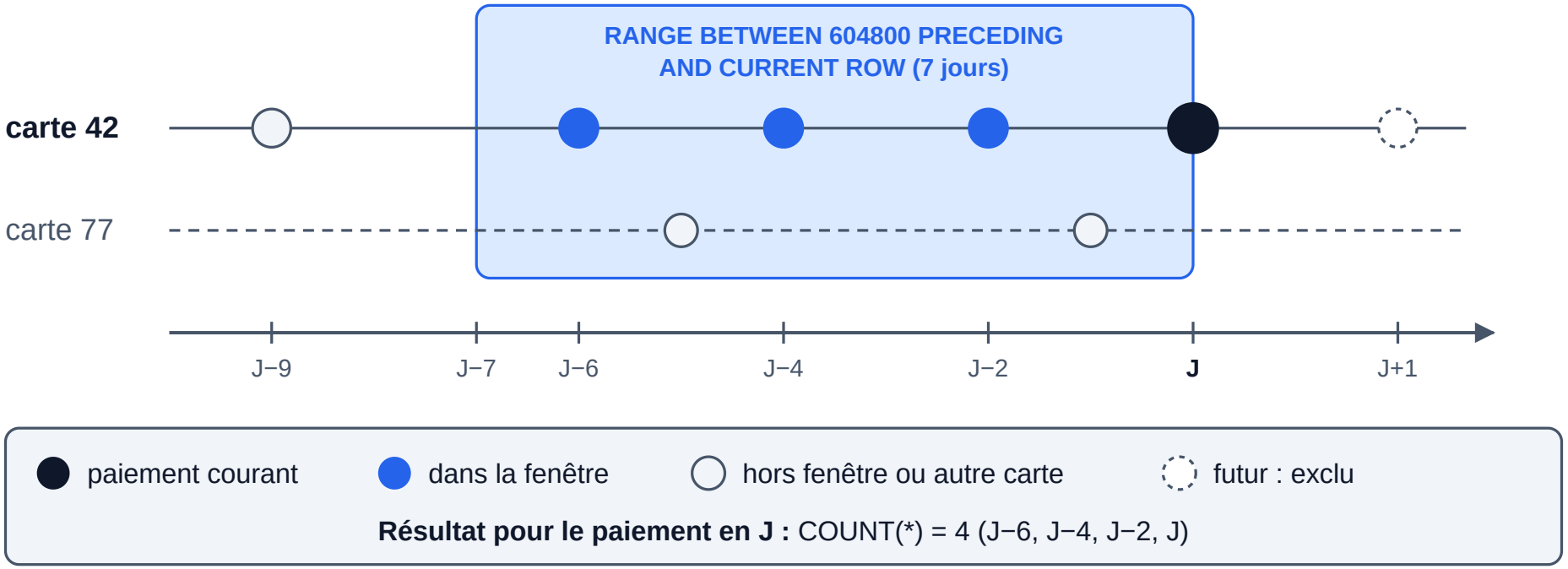


Schéma : pour chaque paiement, la fenêtre remonte de 7 jours sur la même carte (PARTITION BY card_id) et s'arrête à la ligne courante.

Partie 4 : gold, la table de features (suite)

Aides :

- Fenêtre glissante sur le temps : `ORDER BY unix_timestamp(payment_ts) RANGE BETWEEN 604800 PRECEDING AND CURRENT ROW` (7 jours = 604 800 secondes).
- Cartes distinctes sur une fenêtre : `size(collect_set(card_id) OVER (...))`.
- Les paiements PayPal n'ont pas de `card_id` : remplacez-le par une valeur unique par paiement (ex. `coalesce(card_id, -payment_id)`) pour ne pas regrouper tous les paiements PayPal ensemble.
- Les tables `orders` (pays de livraison) et `customers` (pays, date d'inscription) du TP 1 sont dans le même schéma.

Partie 4 : gold, la table de features (suite)

Partie 4 : sources de gold_payment_features

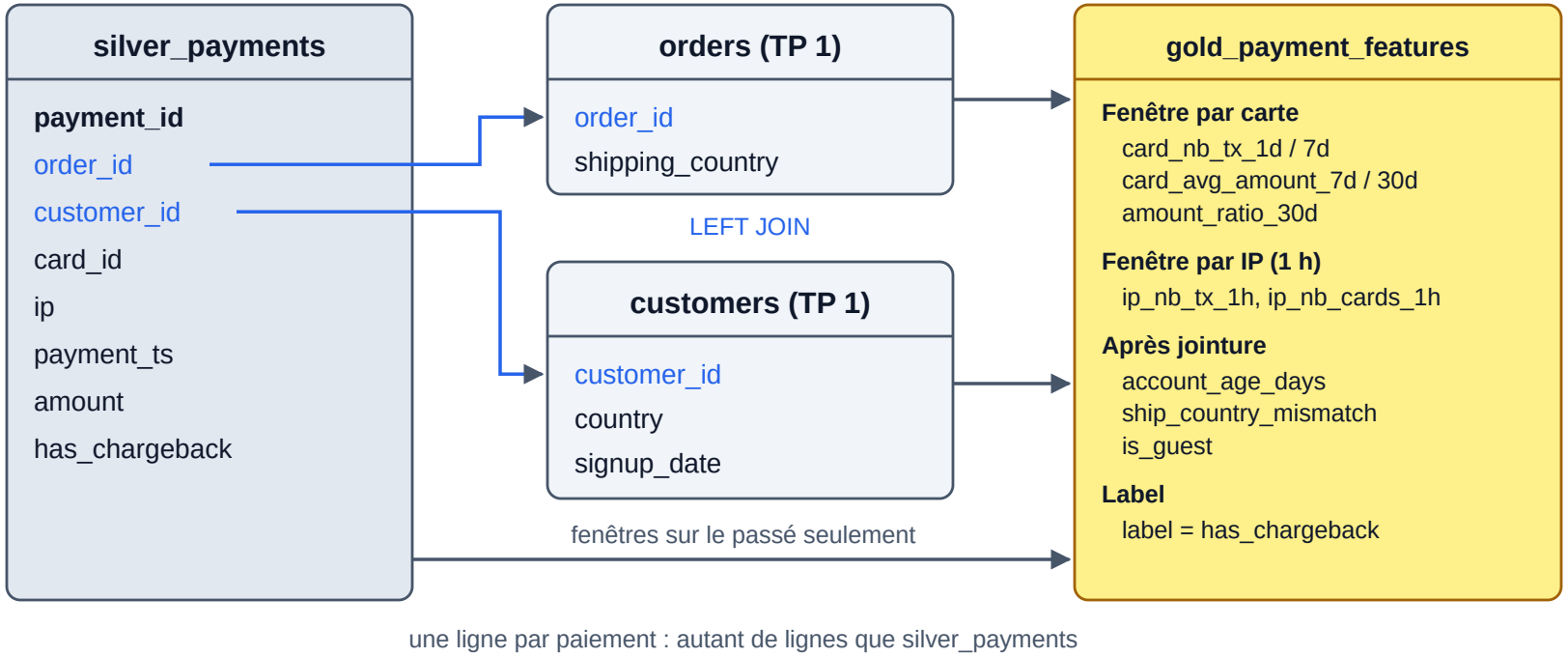


Schéma : d'où vient chaque groupe de features ; les fenêtres se calculent sur silver_payments, les autres après jointure.

Partie 4 : gold, la table de features (suite)

Point de contrôle : autant de lignes que `silver_payments`.

Vérification de bon sens :

```
SELECT label, ROUND(AVG(ip_nb_cards_1h), 2) AS cartes_par_ip, ROUND(AVG(amount_ratio_30d), 2) AS ratio,
       ROUND(AVG(CAST(ship_country_mismatch AS INT)), 3) AS mismatch
FROM workspace.shopflow.gold_payment_features
GROUP BY label;
```

❓ Q8. Quelles features séparent le mieux les fraudes des paiements légitimes d'après cette requête ? À quels scénarios de fraude correspondent-elles ?

❓ Q9. Pourquoi la fenêtre ne doit-elle porter que sur les paiements **précédents** ? Que se passerait-il en production sinon ?

Partie 5 : maintenance

1. Créez volontairement une table fragmentée :

```
(spark.table("workspace.shopflow.silver_payments")
      .repartition(400)
      .write.mode("overwrite").saveAsTable("workspace.shopflow.payments_fragmented"))
```

2. Relevez `numFiles` et `sizeInBytes` avec `DESCRIBE DETAIL`. Chronométrez `SELECT card_country, SUM(amount) FROM ... GROUP BY card_country`.

3. Lancez `OPTIMIZE workspace.shopflow.payments_fragmented;` puis refaites les deux mesures. Regardez la dernière ligne de `DESCRIBE HISTORY` (colonne `operationMetrics`).

4. Lancez `VACUUM workspace.shopflow.payments_fragmented DRY RUN;`.

❓ Q10. Combien de fichiers avant et après `OPTIMIZE` ? Pourquoi les anciens fichiers sont-ils toujours sur le stockage ? Que liste le `DRY RUN`, et pourquoi aucun fichier n'est-il encore supprimable ?

Partie 5 : maintenance (suite)

Si Databricks a déjà regroupé les fichiers à l'écriture (écritures optimisées), notez-le : c'est un comportement de la plateforme à commenter.

Partie 6 : restitution

Chaque binôme présente :

1. Dessinez votre pipeline bronze → silver → gold avec les volumes et les tables.
2. Donnez un exemple concret où le *time travel* vous a servi aujourd'hui.
3. Une feature que vous ajouteriez, et à quel scénario de fraude elle répond.

Si quelque chose bloque

Symptôme	Cause probable	Que faire
`INFINITE_STREAMING_TRIGGER_NOT_SUPPORTED`	Requête de streaming sans `trigger(availableNow=True)`	Utiliser la fonction `ingest` fournie
L'ingestion ne lit rien de nouveau	Les fichiers ont déjà été vus (point de reprise)	Normal ; pour tout recommencer, supprimez la table bronze **et** le dossier `checkpoints/<source>`
`CHECK constraint amount_positive violated`	Une ligne négative arrive dans silver	Votre filtre de quarantaine est incomplet
Le `MERGE` échoue : plusieurs lignes source pour une même cible	Doublons non retirés dans la source du `MERGE`	Dédoublonner avant le `MERGE`
`VERSION AS OF` introuvable	Mauvais numéro de version	Relisez `DESCRIBE HISTORY`
Calcul coupé pour la journée	Quota quotidien de Free Edition dépassé	Reprendre le lendemain ; éviter de relancer en boucle